

ABSTRACT

Large language model (LLM)-based agents increasingly rely on tool use to complete real-world tasks. While existing works evaluate the LLMs’ tool use capability, they largely focus on the final answers yet overlook the detailed tool usage trajectory, i.e., whether tools are selected, parameterized, and ordered correctly. We introduce TRAJECT-Bench, a trajectory-aware benchmark to comprehensively evaluate LLMs’ tool use capability through diverse tasks with fine-grained evaluation metrics. TRAJECT-Bench pairs high-fidelity, executable tools across practical domains with tasks grounded in production-style APIs, and synthesizes trajectories that vary in breadth (parallel calls) and depth (interdependent chains). Besides final accuracy, TRAJECT-Bench also reports trajectory-level diagnostics, including tool selection and argument correctness, and dependency/order satisfaction. Analyses reveal failure modes such as similar tool confusion and parameter-blind selection, and scaling behavior with tool diversity and trajectory length where the bottleneck of transiting from short to mid-length trajectories is revealed, offering actionable guidance for LLMs’ tool use.

1 INTRODUCTION

Large language models (LLMs) increasingly exhibit strong planning and reasoning abilities (Wei et al., 2022; Yao et al., 2023a): they break goals into subproblems, track intermediate states, and arrange multi-step procedures (Huang et al., 2024; Song et al., 2023). Together, these abilities position LLMs as the “brain” of agentic systems, guiding the systems to perform complex, real-world tasks (Wang et al., 2024; He et al., 2024b; Li et al., 2024). Meanwhile, external tools, such as search engines (Jin et al., 2025), production APIs (Li et al., 2023a), and file/OS operations (Packer et al., 2023), function as the agent’s “hands”, extending LLM’s abilities with precise computations, up-to-date information, and concrete actions. Tool-use has already powered agents across domains: travel agents (Chen et al., 2024; Singh et al., 2024) fuse flight and hotel APIs with visa and weather services to assemble feasible itineraries and resolve constraints; and education agents (Chu et al., 2025; Zhang et al., 2024b) retrieve curriculum materials, generate adaptive exercises, grade against rubrics, and track progress across learning platforms.

Despite these developments, proper evaluations of LLMs’ tool-use are essential: they can provide a complete picture of tool-use competence, expose failure modes, and point out concrete directions for improvement. Specifically, the capability to select the right tool(s), determine the correct formats and values of input parameters, conduct multi-step tool utilization, and adapt to unseen tools, makes the core part of evaluating tool-use capability. Benchmarks have been created to test these abilities. For example, Huang et al. (2023) focus on evaluating whether LLMs can determine when to call tools, and Qin et al. (2023); Patil et al. (2024); Patil et al.; Zhuang et al. (2023) evaluate LLMs’ capability in calling proper tools for solving complex queries.

However, significant gaps in tool-use evaluation still persist. First, tool-use trajectory complexity is comparatively underexplored: some existing suites rely on small or simulated tools (Zhuang et al., 2023) and many only test short, low-depth tool trajectories (Qin et al., 2023). However, real agents can possess large tool sets and face complex user queries, so evaluation on a larger executable tool set and trajectories with more tools involved is still needed. Second, the complexity of real user queries is also underrepresented: existing benchmarks often consider straightforward prompts by including the API name directly in the prompt. In contrast, real-world agents may face user queries that consist of indirect language and implicit cues, and the agents need to infer both the choice of the tool and how to set the inputs. Third, most benchmarks still privilege final-answer metrics. For example, Qin et al. (2023) only provide pass rate and win rate of the final answer, and Patil et al. heavily rely on the overall accuracy. In this case, it is hard to track the root cause of the incorrect final answer, which can be caused by various issues such as incorrect tool selection, disorders of tool-use, or incorrect parameterization. This oversight can obscure the evaluation of LLMs’ tool-use capability and cannot differentiate it from other general reasoning capabilities, especially given the observation in (Roberts et al., 2020; Qian et al., 2025b) that LLMs can solve the problem using internal knowledge even when the wrong tools are called.

To bridge these gaps, we present **TRAJECT-Bench**—a benchmark to comprehensively evaluate the LLMs tool-use capability via providing **(1) tool-use trajectories** of different complexities and **(2) user queries of different levels of difficulties** given the same tool-use trajectory. Meanwhile, we also include **(3) evaluation metrics** to evaluate the LLMs’ tool-use capability from diverse perspectives. As summarized in Table 1, we are the first to provide all the comprehensive analysis perspectives for tool-use evaluation compared to existing benchmarks.

During the data construction, to ensure the quality of the data and better align it with real scenarios, on the tool side, we select a diverse suite of over 1,000 high-fidelity tools drawn from various real-world domains (e.g., finance, travel, music, etc.), exposed via production-style APIs. Given these tools, to construct **(1) tool-use trajectories**, we further synthesize task-driven tool-use trajectories that encode different trajectory structures and scales. We consider both parallel and sequential tool calling trajectory structures and a trajectory scale of 3 to 10+ tool counts. Moreover, for each trajectory, we provide **(2) two semantically aligned queries of different query difficulties**: a direct and explicit “simple” version and a naturalistic and indirect “hard” version. Such a data construction procedure allows us to decompose the queries’ difficulty into different perspectives and understand the corresponding specific weaknesses of LLMs.

Based on this data, we evaluate the state-of-the-art models, compare representative tool-selection strategies, and assess agentic tool-use settings beyond individual LLMs. For **(3) the evaluation metrics**, we report trajectory-aware metrics alongside final-answer accuracy: Trajectory Exact-Match and Trajectory Inclusion (whether the required tools are invoked and in the correct order), Tool-Usage (schema constraints, formats, and value checks of the tool’s inputs), and an LLM-judge Trajectory-Satisfy score when gold traces are unavailable. Together, these evaluations treat tool-use as a primary skill, with trajectory-aware metrics that explicitly measure trajectory and query complexity. Our key contributions are summarized as follows:

- **Dataset and tasks.** We introduce high-quality, executable tool suites across various domains and tasks. We model trajectory complexities via different structures (*parallel* and *sequential*) and various tool counts. Queries are aligned with trajectories with different difficulty levels.
- **Evaluation suite.** We evaluate state-of-the-art LLMs and representative tool selection strategies. We also include agentic tool-use in the evaluation suite. Beyond final-task accuracy, we introduce trajectory-aware metrics to better capture tool-use capability.
- **Insights and guidance.** We provide analyses based on our evaluation, and reveal important insights, such as models’ struggling to infer correct tools from indirect queries, the bottleneck of

the transition from short to mid-length trajectories, limitations of retrieval-based selection. These insights can inspire future development of a more precise and reliable tool-use mechanism.

3 DATA CONSTRUCTION

To construct the data, we first select the candidate tools to be included in the benchmark (Section 3.1), and then curate ground truth tool-using trajectories and corresponding queries (Section 3.2).

3.1 TOOL SET CURATION

Following (Qin et al., 2023), we source practical tools from RapidAPI, which contains many APIs for real-world tasks. To keep the effort tractable, we curate data for ten representative domains: travel, mapping, finance, weather, e-commerce, news/media, gaming, email, education, and music, where LLM agents are especially popular and useful¹.

Although thousands of APIs are available from RapidAPI, many of them contain unclear descriptions or parameters, and some are even non-executable. Therefore, we carefully select and revise the APIs based on the following four requirements.

(1) **Executable tools with meaningful outputs.** We validate each tool by executing it across parameter combinations and discard those that error out (details in the Appendix A.1). For the remainder, we use an LLM to summarize outputs and formats, removing tools whose outputs are semantically trivial, that is, do not contribute useful information.

¹More domains can be added following our data-generation pipeline

(2) **Clear, action-oriented tool descriptions.** Because many APIs ship with sparse or vague documentation, we refine API descriptions by combining the original description with empirical I/O observed during validation. For instance, the original description is just ‘Get price (symbol),’ yet empirical calls reveal it enforces a 50-item `page_size` cap and returns `{price, currency, timestamp}`; we merge these behaviors into the clarified description.

(3) **Minimal functional overlap.** We deduplicate identical tools/APIs (e.g., multiple flight-search endpoints) to avoid ambiguity and ensure deterministic trajectory evaluation, keeping a single representative per function. Closely related tools are kept when they introduce distinct parameterizations to increase task complexity. This is done with the help of LLMs, followed by manual verification.

(4) **Controlled tool complexity.** We manually keep tools with strong parameter complexity (number of fields, types, and constraints) and remove some with simpler parameters (e.g., no inputs needed), ensuring the set contains rich tools to stress-test tool-use competence. Finally, we obtain a high-fidelity tool set $\mathcal{T}$. Details and examples of the curated tool set can be found in the Appendix A.1.

3.2 TASK-DRIVEN QUERY GENERATION

To evaluate LLMs’ tool use in realistic settings while keeping the evaluation controllable, we synthesize queries from real-world *task types*, which denote categories/families of tasks that users can adopt the agent for, such as booking flights and arranging stays in the destination (without details of dates or locations). Specifically, as shown in the left side of Figure 1, for each domain we manually collect representative task types (e.g., “real-time trip monitoring & assistance” for travel; “creating math & science study materials” for education), and provide a precise description for each task type. For every task type, we create tool-use trajectories with details and pair them with aligned user queries. By templating from task types, we minimize human efforts in collecting and annotating data, yet keep queries faithful to real use cases. This also enables diverse, controllable, and scalable query–trajectory data across domains.

We consider two basic trajectory structures: (i) *parallel*, where tools operate independently (the choice/execution of one tool does not depend on others); and (ii) *sequential*, where tools form a chain and later steps depend on earlier outputs². Details of trajectory and query construction are provided as follows.

3.2.1 PARALLEL QUERY

Trajectory generation. For each domain, we prompt an LLM to synthesize valid, logical tool-using trajectories from a task-type description and the domain’s available tools. To keep evaluation

²We focus on chain structures to balance data cost and evaluation reliability; richer graph topologies are left for future work.

scalable and comparable, we enforce two rules: (i) the plan must use a specified number of tools (typically 3–10+); and (ii) tools are presented as a parallel-ready set—each call is self-contained (inputs fixed up front) and independent of the others. Parallel trajectories are encoded as unordered sets of tool calls with fully specified inputs. For example, Figure 1 shows that combining hotel and flight search APIs enables end-to-end travel planning.

Query generation. After trajectories are finalized, we pair each trajectory with user queries designed at two difficulty levels:

- **Simple version.** Provide a straightforward and precise instruction that explicitly specifies the need for tools and their key parameters, such as the example in Figure 1 where the simple query gives direct and detailed requests for a travel plan.
- **Hard version.** Present a more challenging, indirect request that conveys the same constraints via natural cues and implications rather than explicit instructions. This mirrors real interactions where users state goals colloquially (e.g., *"hotels with good record"* rather than *"sort hotels by review score"* as shown in Figure 1).

After the generation, both trajectories and queries go over LLM-based automatic validation and human inspection to ensure the quality and reduce ambiguity. By jointly varying (i) the number of tools per trajectory and (ii) the difficulty level of paired queries, we obtain a comprehensive yet well-controlled evaluation of LLMs’ tool-use capabilities.

3.2.2 SEQUENTIAL QUERY

Sequential queries require strong dependencies between tools within the trajectory, which makes it difficult for an LLM to directly propose a correct chain end-to-end. To address this, we first build a **directed tool graph** $G_T = (V, E)$ that captures how tools can feed one another: each tool t in V is a node, and we add a directed edge $t_1 \rightarrow t_2$ to the edge set E , when the information from the output of t_1 can be used as input parameters of t_2 . For instance, as shown in the right side of Figure 1, the IATA codes returned from the GetAllIATA API can be used as the input for airportInfo API to retrieve detailed information of a specific airport, so we connect them.

Given G_T , we generate representative tasks and corresponding tool sequences with a controlled number of tools (e.g., 5 sequences for each number of tools). Unlike the parallel setting, these sequences are manually created as trajectory templates to ensure logical coherence, e.g., $t_1 \rightarrow t_2 \rightarrow \dots \rightarrow t_{n_{traj}}$, where n_{traj} denotes the number of tools involved. Each template explicitly specifies the parameter bindings between adjacent tools, i.e., which output results from t_i populate which input fields of t_{i+1} . For instance, the task sequence in Figure 1 describes a general ‘city- $\hookrightarrow$ hotel location- $\hookrightarrow$ hotel detail’ chain without details such as city name or hotel id.

Similar to the generation of parallel trajectories, we then prompt an LLM to create trajectories with details and corresponding user queries from each template (5 per template), yielding $s_{traj}^{seq} = \{t_1(params) \rightarrow \dots \rightarrow t_{n_{traj}}(params)\}$ together with q_{traj}^{seq} , as shown in Figure 1. All trajectories and their paired queries undergo automated validation and human review to ensure validity, task alignment, and overall quality. More details of the generation process and datasets are shown in the Appendix A.2. This pipeline produces coherent tool chains and tightly aligned queries at varied depths (number of tools involved), enabling scalable and transparent evaluation.

Finally, the whole dataset contains 1228 tools and 5670 queries (details in Appendix A.2).

4 EVALUATIONS ON [TRAJECT-BENCH](#)

We comprehensively evaluate LLMs’ agentic tool-use on [TRAJECT-Bench](#), focusing on key research questions that highlight its benefits and potential:

RQ1. When and why do LLMs succeed or fail at tool-use?

RQ2. Can retrieval-augmented selection improve tool selection and parametrization?

RQ3. Do agentic methods, including training and inference, improve tool-use capability?

4.1 EVALUATION SETTINGS

Test (query) methods. When evaluating LLM’s own capability in calling proper tools to solve queries, we mainly adopt the direct query where user queries and tools are provided to the model. We provide additional results using Chain-of Thought in Appendix C.

Tool selection strategy. Unlike existing benchmarks (Qin et al., 2023; Huang et al., 2023) ship only retrieved tools (not the full tool set) during evaluation, we treat the tool selection strategy as a crucial part of tool-using, especially when the tool set is large and exceeds the model’s context window. We evaluate three common tool selection strategies (labels shown in Table 4): *all*—provide the full tool set in context; *domain*—provide only tools from the query’s domain; and *retrieval*—retrieve a subset based on the query and tool descriptions. For retrieval, we test two widely used embedding models, *all-MiniLM-L6-v2* (Solatorio, 2024) and *bge-large-en-v1.5* (Xiao et al., 2023), plus a tool-specific retriever, *ToolBench-IR* (Qin et al., 2023). Unless noted, we retrieve 20 tools by default.

Test models. We test on a wide range of state-of-the-art LLMs: Claude family, including Claude-3.7 (ant, 2024) and Claude-4 (ant, 2025); Gemini family, including Gemini-2.5-pro and Gemini-2.5-flash; GPT family, including o4-mini and gpt-oss-120B; Qwen3-235b-A22B, the latest generation of LLMs in the Qwen series; DeepSeek-V3.1, the latest DeepSeek model; and Kimi-k2, the latest MoE model from MoonShot AI.

Agentic evaluation. Despite the tool-using capability of models themselves, one highlight of our benchmark is the evaluation of models’ agentic tool-using capability. We consider two agentic capabilities. The first one is the capability specifically trained in the models. The SOTA LLMs, such as Claude, Gemini, Deepseek, and Kimi-k2, all include tool-using in the training and directly support tool-calling, e.g., the “tools” input in `client.messages.create` of Claude APIs, and do not need to additionally list them in the context of the prompt. Therefore, we provide evaluations on these models’ internal tool-using capability and compare with the results of providing tools as context. More details can be found in Appendix B.

We also evaluate LLM agents for tool-use. Given diverse designs, we focus on ReAct (Yao et al., 2023b)—a foundational blueprint for many real agents (e.g., MetaGPT (Hong et al., 2024)). We pair ReAct with domain-specific and retrieval-based tool selection. Because ReAct reasons and executes over multiple turns, we test two retrieval modes: *static* (retrieve once from the query) and *dynamic* (retrieve at each reasoning/execution turn).

Metrics. Unlike existing benchmarks, which mainly focus on the final performance, we consider two categories of metrics: trajectory-aware metrics and final performance metrics. For the trajectory-aware metrics, we consider (1) Exact match (EM), which compares the predicted tool-using trajectory and the ground-truth ones to check if the predicted tools (names, not parameters) are exactly the same as the ground truth ones; (2) Inclusion, which measures what proportion of ground truth tools is included in the predicted tool trajectory; (3) Tool Usage (Usage), which checks if the predicted tool parameters match the ground truth ones; and (4) trajectory satisfaction (Traj-satisfy), where we prompt an LLM judge (Claude-4 by default) to determine to what extent a predicted trajectory can solve the user query, and by using this metric we mimic a real-world scenario when the ground truth is not available. For the final performance metric, we report (5) Acc, which measures if the predicted final answer matches the ground truth answer by prompting an LLM judge. For the retrieval-based methods, we include the (6) retrieval rate, which measures what proportion of ground truth tools are retrieved. Codes and data are available on <https://anonymous.4open.science/r/ToolData-public-2565>.

4.2 RQ1. WHEN AND WHY DO LLMs SUCCEED OR FAIL AT TOOL-USE?

We first evaluate the tool-use capability of individual LLMs. For each evaluating query, we feed the LLM both the query and the available tools as the context. Since the full tool set is beyond the length of the model’s context window, we focus on tools related to the same domain of the query, e.g., we only provide Travel tools for a query in the Travel domain. We report metrics mentioned in Section 4.1 in Table 2 for parallel data and Table 3 for sequential data.

Overall tool-use performance analysis. Generally, we observe that Gemini-2.5-pro performs best on simple queries, while Claude-4 ties with it on the hard versions. Less capable models, such as Claude-3.7 and Gemini-2.5-flash lag behind. The LLM-judge Traj-Satisfy score also tracks EM closely (e.g., Claude-4: 8.549 $\leftrightarrow$ 0.846 on simple; 4.882 $\leftrightarrow$ 0.445 on hard), indicating the judge is an effective proxy for EM.

In addition, we make the following key observations. (a) Most models perform well on the simple versions but struggle on the hard versions. For most metrics on most models, we observe a clear gap between simple and hard; for example, EM for Claude-4 drops from 0.846 $\rightarrow$ 0.445 and for

Gemini-2.5-pro from 0.851 $\rightarrow$ 0.442 under direct prompting. This suggests that when tool choices and constraints must be inferred from indirect cues, models often miss the exact tool set and parameters, and future efforts should focus more on these more complex cases. (b) For nearly every model, *parallel-simple* obtains better performance than *sequential*. For example, the EM, Inclusion, and Usage of Gemini-2.5-pro (0.851, 0.854, 0.835) on simple parallel queries are higher than those on sequential queries (0.807, 0.821, 0.809), suggesting that inter-step dependencies and ordering introduce additional challenges for both tool selection and parameter determination. This motivates future tool-use training and inference methods that explicitly model dependency and order. (c) Inclusion typically exceeds EM, especially on hard parallel and for weaker models. For most hard cases, there is a pronounced Inclusion-EM gap—e.g., 0.135 vs. 0.554 for Claude-3.7 and 0.216 vs. 0.538 for Gemini-2.5-flash—indicating that models fail to recover the complete set.

Tool-use scaling analysis. Since the user queries can be very complex in practice and can involve many tools, we analyze the model’s tool-use capability when the number of tools in the (ground truth) trajectory increases. As an illustration, we present the results on both simple and hard queries on models from different families in Figure 2, where the x-axis is the number of tools and the y-axis is the metric EM. As the number of tools in a trajectory grows, all models experience a clear decline in performance, with the steepest drop occurring between three and five tools. Among the evaluated models, Claude-4 and DeepSeek show the strongest generalization, maintaining relatively stable performance even at longer tool chains, whereas o4-mini and Kimi-k2 collapse sharply beyond seven tools. These results reveal that **scaling tool use is a universal challenge**, where **the main bottleneck lies in the transition from short to mid-length trajectories**, and underline the importance of improving models’ long-horizon tool-use and error recovery capabilities.

Deeper analysis on tool-use failures. We further take a look at failed cases to identify common failure patterns in tool use, and detailed examples are presented in Appendix D. (a) **Similar tool confusion.** This refers to the cases when models face tools that are partially overlapping capabilities but distinct scopes, inputs/outputs, or constraints (e.g., confusing between Spotify: Search vs. YouTube Music: Search). Distinguishing among these tools requires a more delicate understanding of the tools’ exact functionalities, parameters, outputs, etc. Existing models need more improvement to obtain a precise tool match facing such distractions. (b) **Parameter-blind tool selection & usage.** In some cases, models overlook tool parameters (values, formats) when selecting tools, relying primarily on tool descriptions. This can lead to selecting tools with wrong parameters, cascading failures in downstream steps, and degraded end-task accuracy. We also notice that this failure is much rarer for Kimi-k2, suggesting that delicate agentic tool training can effectively improve this problem. (c) **Redundant tool calling.** This refers to cases where the model invokes more tools than necessary. We observe two forms of redundancy: (i) *related-but-unhelpful calls*—e.g., for a query about airport information in Zurich, the model additionally calls “Get All IATA airport codes,” which returns every code irrespective of the city name provided in the query; and (ii) *unrelated calls*—e.g., invoking railway tools when the task asks only for flight information. The first pattern likely reflects a conservative “cover-all-bases” strategy, whereas the second is more consistent with hallucination or weak intent grounding. Since redundant calls can cause inflated latency and cost and even errors, precise tool-calling trajectory data should be included in training for improvement, and careful validation can also be adopted. (d) **Struggle to infer intents from hard queries.** When facing indirect queries, models often misinterpret the user’s intent, leading to entirely irrelevant tool selections.

Together, these failure patterns show that models struggle with accurate tool choice and parameter use in complex scenarios, underscoring the need for better training and inference strategies.

4.3 RQ2. CAN RETRIEVAL-AUGMENTED SELECTION IMPROVE TOOL SELECTION AND PARAMETRIZATION?

Since tool sets can be large and diverse in practice, it is preferable to first narrow them to a smaller, more relevant subset to avoid adding irrelevant tools to the context. Retrieval-based strategy is a popular way to achieve this, and we evaluate different embedding models and different retrieval pools in experiments to find out to what extent retrieval can help. The results in Table 4 suggest two clear findings. (a) **When the retrieval pool is already restricted to domain-related tools, retrieval adds little benefit for simple queries.** For both Claude-3.7 and Claude-4, EM and accuracy remain nearly the same across embedding models, with ToolBench-IR showing only marginal improvements. (b) **Retrieval becomes a severe bottleneck for hard queries.** We notice that the retrieval rate significantly drops (merely over 50% for most models) and all the performance metrics drop sharply compared with the non-retrieval cases. The core issue may be that retrievers heavily rely on semantic similarities and fail to capture underlying intents and steps from implicit queries. Therefore, they struggle to correctly identify necessary tools, causing cascading failures in tool selection and parameterization. Together, these findings reveal limitations of retrieval-based tool selection methods and call for a better strategy.

4.4 RQ3. DO AGENTIC METHODS, INCLUDING TRAINING AND INFERENCE, IMPROVE TOOL-USE CAPABILITY?

Despite the individual model’s capability of using tool information as the context, agentic methods are also developed in existing literature, including two major categories: one is the agentic tool-use training undergone by most SOTA models (Team et al., 2025; Yang et al., 2025), and the other is to design LLM agents to adopt tools (Yao et al., 2023b). For the first category, we evaluate four representative models that inherently support tool-use, and show results in Table 5, where we also provide results of context-based evaluation as a baseline. Table 5 shows that the agentic tool-use capability is similar to the context-based baseline for most models on both simple and hard queries.

For the second category, we evaluate Claude models on both parallel and sequential queries by combining retrieval with ReAct. We consider two settings: a static mode, where a subset of tools is retrieved once based on the user query, and a dynamic mode, where retrieval occurs before each thought and action. ToolLM-IR is used as the retrieval backbone in all experiments. The results in Tables 6 and 7 show that ReAct consistently improves tool-use performance compared with individual models in Table 2. For example, on parallel hard queries, Claude-4 improves from 0.445 EM (Table 2) to 0.463 EM with ReAct (Table 6), and the performance is further boosted via dynamic retrieval to 0.473 EM. Similarly, Claude-3.7 gains from 0.135 EM to 0.186 EM with domain tools and 0.296 EM under dynamic ReAct. These show that **iterative calling tools based on execution results offer a stronger basis for accurate tool retrieval and usage.**

Overall, these findings suggest that agentic tool learning and inference serve as an effective and robust way to improve a model’s tool-use capability.

5 CONCLUSION

This work introduces a comprehensive tool-use benchmark that focuses on the tool-use trajectory. By constructing a large and diverse executable tool suite, modeling trajectories of varying structures and scales, and pairing them with user queries of different difficulty levels, we provide a realistic and rigorous setting for evaluation. Our trajectory-aware metrics go beyond final-answer accuracy, enabling a clearer understanding of where and why LLMs succeed or fail in tool use. Through comprehensive evaluation and analysis, our benchmark not only highlights the current strengths and limitations of state-of-the-art models but also offers actionable insights for improving agentic tool-use capabilities. We hope this benchmark establishes a foundation for systematic progress in developing LLMs that can reliably plan, select, and execute tools in complex, real-world scenarios.